Draft Analysis Report

Lauren: I was in charge of the diagram for the events/registration section. I first created the table event_reservation table that takes care of all the information that is provided when booking, like start time, end time, estimated number of guests, etc. This table also needed to contain customerId, hotelId, and facilityId just in case any of the information in those tables changed. I then created an event table that established an eventId and a name. I included the primary key from the event_reservation table, eventReservationId, to connect the event to the necessary reservation requirements. I also needed to establish what room would be used for the event, since one event can use many rooms at the same time. I created an event_room table with eventId and roomId as the foreign keys and eventRoomId as the primary key.

The two assumptions/rules I made were 1) the event table only has day-of information while event_reservation has details provided in advance; 2) one room can only have one event going on at the same time (it would be an edge case if one room can have multiple events going on at the same time).

Yixuan: I worked on the physical infrastructure part through the hierarchy of hotel complex, building, wing, and room. Each hotel complex contains multiple buildings, each building contains multiple wings, and each wing contains multiple rooms. These entities are connected though one-to-many relationships using foreign keys. Room numbers are unique within each wing, but could be repeated across the hotel complex. As for the facility table, it separately stores information about services including their types and locations. Facilities are associated with a hotel complex and a specific building. The room table is one of the core entities within the whole ERD. It stores information for room management. The roomType attribute tells what the room is used for, which could be not only sleeping/meeting, but also flexible, suite, or ballroom. Since a room may contain multiple beds of different types and sizes, I created a separate bed table to record bed specifications through a one-to-many relationship. I also built a room_adjacency table to represent physical connections between rooms. This table contains two FKs that work as PKs referencing the room table, allowing a room to be adjacent to multiple other rooms while recording the connection wall type and whether private access doors are available. The room_status_record table tracks changes in room availability over time, including cleaning, maintenance and renovation. Each status record is associated with a specific room and includes a status type and start and end types. Finally, I made a sleeping_reservation table apart from the event_reservation to store the customer's booking requirements, which is associated with the customer and the hotel_complex table through FKs. The sleeping reservation table is also optionally referenced to the event reservation table so that the hotel can identify accommodations. The hotel_stay_record table connects a sleeping reservation to the actual room occupied by a customer.

My assumptions/rules are: 1) Room numbers are unique within each wing. I assume that the floor designation is already included in the room number, and a separate Floor table is unnecessary. 2) A divisible room and all its subrooms are stored as separate records in the room table, each with a unique roomId. A self-referencing parentRoomId identifies the larger room to which each subroom belongs. For example, ballroom 101 may be divided into Rooms 102, 103,

and 104. All four rooms are recorded individually, with ballroom 101 identified as the parent room. Overlapping reservations involving a parent ballroom and its subrooms must be prevented to avoid booking the same physical space. 3) Not every room contains a bed, since some rooms are intended exclusively for meetings. Therefore, a room may have zero or multiple associated bed records. 4) Each row in hotel_stay_record represents one customer's continuous occupancy of one room. If a customer changes rooms, a new stay record is created, while the records may remain linked to the same Sleeping Reservation. 5) Outdoor spaces such as courtyards and pool patios are treated as meeting rooms when they are used for events.

Carol: Everything that can be billed for are the services, events, and overnight stays. This is represented by the total_bill table connected to the service_charges, event_reservation, and sleeping_reservation tables. The service_charges table then connects with service to identify each purchase and has attributes to price it accurately. The prices of the rooms is an attribute of the room itself, which is connected to the event and sleeping reservation tables. The bill to the reservations are one to one but the bill to services is one to many since there can be many services ordered. The billed_party table then identifies who is paying. This table includes all the information needed to get a payment as attributes and also includes splitting which makes the relationship one to many.

Some assumptions we have here is that each bill is for one reservation, be it an event or overnight reservation, but each reservation can have multiple service charges associated with it. We assume that the billed people would only split by the charges (services, reservations) or by percent, and even if they aren't staying at the hotel, they are a customer who has a customerId.

Sua: I was responsible for the tables related to people and card access: customer, organization, customer_qualifications, guest_messages, customer_card, customer_scan, and card_reader tables. Customer is a central repository for individuals including guests, hosts, and organization representatives, which table stores contact information, optional organizational ties, and whether they are open with sharing their whereabouts. The table organization stores corporate and group entities hosting events or sponsoring guest stays. A customer may belong to an organization; independent guests leave this foreign key NULL. The table customer_qualifications tracks creditworthiness, deposit requirements and past behavioral flexibility ratings used by staff to make pricing and deposit decisions. The table guest_messages manages voicemail, front-dest messages, and availability updates left by or intended for guests. The table customer_card represents physical smart cards with embedded PINs issued to guests and staff upon check-in for room or facility access and location identification. The table card_reader is a catalogue of physical card scanning hardware positioned at doors. The table customer_scan is the bridging table between customer_card and card_reader, which records entry/exit timestamps, card PIN number, and reader location as the card is scanned.

Assumptions: 1) A customer can receive multiple cards over successive units. 2) Physical readers contain separate entry and exit slots with the same cardReaderId, captured by the readDirection attribute in the customer_scan table.

Team Statement

Our team participated in a collaborative work on our ERD of Last Resorts Hotel, demonstrating shared understanding and planning. First, we collectively analyzed the system requirements and brainstormed a comprehensive list of potential database tables. From this analysis, we grouped the system into four major functional categories: physical infrastructure, events/reservations, billing/services, and people and access. By establishing a shared conceptual model, every team member gained a clear understanding of how individual database components interact across the whole system.

To ensure an equal distribution of workload, primary responsibility for each functional category was assigned to each team member. Each member's responsibility includes establishing Crow's Foot notation and cardinality constraints, drafting explanatory notes and rationale, and reviewing adjacent entities to maintain consistency across categories.

For flexibility, our team had both real-time group sessions and asynchronous independent work. We held scheduled online meetings to review individual contributions, resolve complex entity relationships, and collect different perspectives and ideas into unification. Additionally, an active group chat served as a channel for open communication and quick team updates.